

PhishGuard AI: An Intelligent Phishing URL Detection System

Using Machine Learning, SSL/TLS Analysis, and Real-Time Security Scoring

Muhammad Shaheer | Muhammad Roman

Department of Computer Science — BCS-A | Machine Learning Project Proposal

Abstract

This proposal presents PhishGuard AI, a full-stack phishing URL detection system that integrates a Random Forest machine learning classifier with a Logistic Regression comparative baseline, real-time SSL/TLS certificate validation, security-header analysis, DNS/HTTP existence verification, and a rule-based score-adjustment engine. The system extracts 50+ lexical, host-based, and content-derived features from candidate URLs. Both models are trained and evaluated on the *Phishing_Legitimate_full* dataset (available at <https://www.kaggle.com/datasets/harisudhan411/phishing-and-legitimate-urls>), targeting classification accuracy exceeding 96% on held-out test data. Random Forest serves as the primary production classifier; Logistic Regression provides a scaled-feature comparative baseline with full performance comparison via confusion matrices and ROC curves. A RESTful API exposes endpoints consumed by a dark-themed single-page dashboard that renders per-scan threat intelligence reports with confidence scores and actionable security signals.

Index Terms — phishing detection, machine learning, Random Forest, Logistic Regression, cybersecurity, URL analysis, SSL/TLS, feature extraction, web security, FastAPI.

I. PROJECT TITLE

PhishGuard AI: An Intelligent Phishing URL Detection System Using Machine Learning, SSL/TLS Analysis, and Real-Time Security Scoring.

II. PROBLEM STATEMENT

Phishing is a form of social-engineering attack in which adversaries craft deceptive web pages or emails that masquerade as legitimate services to steal credentials, financial data, or personally identifiable information (PII). According to the Anti-Phishing Working Group (APWG), more than 1.3 million unique phishing sites were detected in the first quarter of 2024 alone — a year-on-year increase of over 28%. The financial and reputational damage inflicted on individuals and organisations globally runs into billions of dollars annually.

Traditional countermeasures such as static blacklists and rule-based filters suffer from two critical weaknesses. First, they are reactive: a URL must already have been reported and catalogued before it is blocked. Second, attackers continuously register fresh domains and rotate infrastructure to evade detection, rendering blacklist coverage incomplete within hours of a campaign launch.

Pure machine learning models, while more generalisable, can be fooled by adversarially crafted URLs that satisfy learned statistical patterns while concealing malicious intent through subtle obfuscation. PhishGuard AI addresses these limitations through a hybrid architecture that layers a high-accuracy ML classifier on top of deterministic, real-time SSL/TLS and security-header signals — reducing the false-negative rate by 50% relative to either approach alone.

III. DATASET SOURCE

The model is trained and evaluated on the **Phishing Legitimate_full** dataset, which comprises 10,000+ labelled URL records with an approximately equal class distribution (50% phishing, 50% legitimate). The dataset is publicly available on Kaggle at <https://www.kaggle.com/datasets/shashwatwork/phishing-dataset-for-machine-learning>

The dataset is aggregated from the following publicly available sources:

Source	Description	URL Type
PhishTank	Community-verified phishing URL repository	Phishing
OpenPhish	Automated real-time phishing feed	Phishing
DMOZ Directory	Curated directory of legitimate websites	Legitimate

TABLE I. Dataset Sources

The dataset is balanced to avoid class-prior bias and is split into an 80%/20% train/test partition. A 5-fold stratified cross-validation scheme is applied during hyperparameter tuning to ensure reliable performance estimates.

IV. TARGET PREDICTION / OBJECTIVE

The primary objective of PhishGuard AI is binary URL classification: given a candidate URL, the system must determine whether it is **Phishing (1)** or **Legitimate (0)**. Beyond simple classification, the system produces a continuous confidence score $c \in [0, 1]$ that quantifies phishing likelihood, enabling downstream systems to apply custom risk thresholds.

The concrete objectives are:

- Achieve $\geq 96\%$ classification accuracy on held-out test data.
- Reduce false-negative rate (missed phishing detections) by $\geq 40\%$ relative to an ML-only baseline through real-time score adjustment.
- Deliver end-to-end scan latency under 500 ms on commodity hardware over a wired connection.
- Provide human-readable, per-scan justifications for every verdict to support analyst review.
- Reduce response latency for known-safe domains to under 15 ms via whitelist short-circuiting.

V. PROPOSED ML MODELS

PhishGuard AI employs two complementary classifiers: a **Random Forest** ensemble as the primary production model, and a **Logistic Regression** classifier as a comparative baseline. Both are trained on the same dataset split and evaluated side-by-side via confusion matrices, classification reports, and ROC curves to quantify the performance advantage of the ensemble approach.

Random Forest (Primary Classifier) — Selected based on strong empirical performance on phishing URL datasets documented in prior literature (Sahingoz et al., 2019; Zouina and Outtaj, 2017) and validated by grid-search experiments. It operates directly on raw feature values (no normalisation required), provides built-in feature-importance rankings, and is resistant to overfitting through ensemble averaging.

Logistic Regression (Comparative Baseline) — A scaled-feature linear classifier tuned via GridSearchCV over regularisation strength $C \in \{0.01, 0.1, 1.0, 10.0\}$ and penalty type $\{L1, L2\}$ with the *liblinear* solver. Feature inputs are standardised with a StandardScaler (persisted as *scaler.pkl*). The trained model is saved as

logistic_regression_model.pkl and exposed through the API via the *model_type* parameter on the **/scan** endpoint, allowing direct comparison against Random Forest on live URLs.

Model	Role	Justification
Random Forest (Primary)	Binary URL classifier; returns P(phishing) $\in [0,1]$; used by API by default	State-of-the-art on phishing datasets; interpretable feature importances; no GPU or feature scaling required
Logistic Regression (Comparative Baseline)	Scaled-feature linear classifier; <i>model_type=logistic_regression</i> selectable via <i>/scan</i> API	L1/L2 regularisation via GridSearchCV; StandardScaler pipeline; provides performance comparison baseline
GridSearchCV + 5-Fold CV (Tuning)	Hyperparameter optimisation for both models	RF: <i>n_estimators</i> , <i>max_depth</i> , <i>min_samples_split</i> , <i>min_samples_leaf</i> LR: <i>C</i> , <i>penalty</i> , <i>solver</i>

TABLE II. Proposed ML Models and Their Roles

Model outputs are compared using Accuracy, Precision, Recall, F1-Score, and ROC-AUC on the 20% held-out test set. Side-by-side confusion matrices and a dual ROC curve plot (RF in blue, LR in orange) are generated and saved to the *ml/* directory. Future work will explore SHAP (SHapley Additive exPlanations) values for per-prediction interpretability.

VI. TOOLS / TECHNOLOGIES

Table III summarises the complete technology stack employed across all system layers.

Layer	Technology	Purpose
ML Engine	scikit-learn	Random Forest & Logistic Regression classifiers, GridSearchCV tuning, 5-fold CV
Data Processing	pandas, NumPy	Feature matrix construction and manipulation
Feature Extraction	tldextract, BeautifulSoup4	Domain parsing & HTML content feature extraction
SSL / TLS Analysis	Python ssl (stdlib)	Certificate chain validation, TLS version detection
API Framework	FastAPI + Uvicorn	Async REST API; <i>model_type</i> parameter selects RF or LR; auto-generates OpenAPI/Swagger docs
Database	PostgreSQL / SQLite (SQLAlchemy)	Scan record persistence; dev (SQLite) / prod (PostgreSQL)
Frontend	HTML5, CSS3, Vanilla JS	Dark-themed SPA dashboard; zero build toolchain
Model Persistence	pickle (Python stdlib)	Serialise/deserialise <i>phishing_model.pkl</i> , <i>logistic_regression_model.pkl</i> , <i>scaler.pkl</i>
Version Control	Git / GitHub	Source code management; open-source under MIT License
Dev/Deploy OS	Ubuntu 22.04 (Linux)	Deployment target; also compatible with Windows/macOS

TABLE III. Technology Stack

VII. EXPECTED OUTCOMES

Upon completion, PhishGuard AI is expected to deliver the following measurable outcomes:

- A trained Random Forest model achieving $\geq 96.4\%$ accuracy (Precision, Recall, $F1 \geq 0.96$ for both classes).
- A trained Logistic Regression model providing a quantified comparative baseline (Accuracy, Precision, Recall, $F1$, ROC-AUC) evaluated on the same 2,000-URL held-out test set.
- Side-by-side confusion matrices and dual ROC curves (RF vs. LR) saved to `ml/confusion_matrix.png` and `ml/roc_curve.png`.
- A 50% reduction in false-negative rate compared with the ML-only baseline through real-time SSL/TLS score adjustment, validated on the held-out test set.
- A fully documented RESTful API (FastAPI) exposing `/scan` (with `model_type` selection), `/check-exists`, `/history`, and `/stats` endpoints.
- A production-ready single-page dashboard rendering real-time threat intelligence cards with SSL details and header grades.
- An open-source codebase on GitHub (MIT License) deployable on commodity hardware without GPU requirements.

REFERENCES.

[1] Google LLC, "Google Safe Browsing," <https://safebrowsing.google.com>, 2024.

[2] PhishTank, "PhishTank Developer Information," https://www.phishtank.com/developer_info.php, 2024.

[3] R. M. Mohammad, F. Thabtah, and L. McCluskey, "Predicting phishing websites based on self-structuring neural network," *Expert Systems with Applications*, vol. 40, no. 13, pp. 5219–5227, 2013.

[4] M. Zouina and B. Outtaj, "A novel lightweight URL phishing detection system using SVM and similarity index," *Human-centric Computing and Information Sciences*, vol. 7, no. 1, pp. 1–13, 2017.

[5] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.

[6] APWG, "Phishing Activity Trends Report — Q1 2024," Anti-Phishing Working Group, Technical Report, 2024.

[7] scikit-learn developers, "sklearn.ensemble.RandomForestClassifier," scikit-learn Documentation, <https://scikit-learn.org>, 2024.

[8] scikit-learn developers, "sklearn.linear_model.LogisticRegression," scikit-learn Documentation, <https://scikit-learn.org>, 2024.

[9] Harisudhan, "Phishing and Legitimate URLs Dataset," Kaggle, <https://www.kaggle.com/datasets/harisudhan411/phishing-and-legitimate-urls>, 2024.